

Relatório – Lista 4

Disciplina: Estrutura Dados

Alunas: Ana Julia da Silva e Lisânia Soares Miranda

1. Representação da Trie

O tamanho do alfabeto escolhido foi de 36 caracteres, composto por 26 letras (de 'a' a 'z') e 10 números (de 0 a 9). Os quais foram mapeados utilizando sua posição na tabela ASCII. Uma vez transformados para o padrão de pesquisa do sistema, as letras já estão minúsculas, não temos espaços e os números foram agregados. Para decidir suas posições no array de filhos, usamos métodos separados para as letras e números:

- **Letras:** calculamos o index das letras subtraindo o valor numérico do caractere (definido pela tabela ASCII) pelo valor numérico de 'a'. Como 'a' tem o menor valor dentre as letras minúsculas (97), o index é decidido pela diferença entre o valor do caractere de índice maior e o de menor ('a'). Dessa forma, o caractere 'a' assume a posição [0], o 'b' a posição [1], até o caractere 'z', que ocupa a posição [25].
- **Números:** já para os números, o índice do caractere '0' na tabela ASCII foi usado como referência. Contudo, aqui também levamos em consideração os 26 caracteres com index anteriormente alocados. Dessa forma, o index dos números é calculado a partir da diferença entre o valor de '0' e o caractere numérico somado ao número 26. Assim, o caractere '0' ocupa a posição [26], e o '9' a posição [35].

A respeito da conversão para a chave de busca, foram considerados apenas os caracteres não vazios (ignorando os espaços). Se o caractere for não vazio, verificamos se seus valores na tabela ASCII correspondem as letras maiúsculas entre 'A' e 'Z', se sim, então convertemos o caractere para o seu correspondente minúsculo somando 32 a seu valor (32 é a diferença entre qualquer letra maiúscula e sua correspondente minúscula). Se o caractere não vazio for um número, nós apenas o concatenamos à variável transformada, sem passar por nenhuma conversão.

2. Funcionamento dos métodos

- **Insert:** Ao receber um título, começamos a percorrer nossa Trie pela raiz, checando se cada um dos caracteres do título está na lista de filhos do nó atual. Começando pelo primeiro caractere, checamos se ele está na lista de filhos da raiz, se estiver, movemos para o nó que o represente (e agora esse é nosso nó atual), se não estiver, adicionamos o caractere a lista de filhos e movemos para ele. Isso é feito iterativamente até que se chegue a última posição da chave de busca.
- **Contains:** Percorremos nossa Trie verificando se cada caractere da chave de busca está contido em nossa árvore. Fazemos isso checando a lista de filhos uma a uma de acordo com que caminhamos entre os nós já existentes. Se durante a busca nos depararmos com algum caractere que não está na lista de filhos do nó atual, dizemos que o título não está contido na Trie.
- **Autocomplete:** No autocomplete, o processo inicial se parece um pouco com o **contains**: para cada caractere do prefixo verificamos se ele está contido na trie, e, se algum não estiver, nós retornamos uma lista vazia. Se o prefixo digitado estiver contido da Trie, pegamos todos os jogos correspondentes a esse prefixo e os ordenamos com base na popularidade. Por fim, retornamos apenas os *k* primeiros jogos.

3. Análise de custo

- **Insert:** o custo computacional dessa função é $O(L)$, onde L é o tamanho do título após conversão para chave de busca. Isso se deve ao loop interno da função que sempre percorre cada um dos L caracteres do título em todos os casos. Quando o caractere não está na lista de filhos do nó atual, também temos o custo de criar um novo nó e adicionar na posição id , mas que não impacta significativamente no custo total.
- **Contains:** o custo computacional dessa função é proporcional a quantidade de caracteres do título convertido que estão contidos na Trie. No pior dos casos, quando a chave de busca está por inteira na Trie, nossa aplicação opera em $O(L)$.
- **Autocomplete:** já a função **autocomplete** envolve o custo computacional de outras duas funções: **collectGames** e **sortResults**. Em **autocomplete**, temos o custo do loop interno que percorre todo o prefixo passado como parâmetro e transformado em chave de busca. No pior dos casos, todo o prefixo estará contido da Trie, e percorreremos todos os seus p caracteres no loop. Assim, essa primeira parte do código tem custo $O(p)$. Depois de encontrar o prefixo, nós coletamos todos os jogos com o prefixo presente no título. Dessa forma, a função **collectGames** percorre cada um dos nós na subárvore do prefixo, tendo o custo $O(s)$. Por fim, a função **sortResults** ordena através do algoritmo Insertion Sort todos os r jogos encontrados para esse prefixo. No pior dos casos, o vetor estará completamente invertido em relação a ordem desejada, e, assim, tem custo $O(r^2)$. Enquanto, no melhor dos casos (em que o vetor está quase ordenado) o custo se torna $O(r)$. Afinal, o custo computacional geral da função depende do máximo entre p , s e r . Assim, a função opera em $O(\max(p, s, r))$.

4. Comparação e uso de memória

Considerando a solução ingênua como um array de objetos do tipo *game*, o custo da busca pelo título seria dependente da quantidade de jogos existentes na base de dados. Além disso, para verificar se um prefixo está contido em um título, teríamos que iterar sobre cada caractere do título enquanto o comparamos com o prefixo. Assim, considerando n o número de jogos do sistema e p o número de caracteres do prefixo, no pior dos casos o sistema percorreria os n jogos e compararia os p caracteres do prefixo para cada um deles, operando assim em $O(n \times p)$. Enquanto nossa solução com a Trie opera em $O(\max(p, s, r))$, independentemente se temos 10 ou 1.000.000 jogos na base de dados.

Se armazenamos os jogos apenas em um array, consumiremos n espaços de memória, um para cada jogo. Considerando a aplicação de uma Trie, o sistema está alocando, a cada nó, um array com 36 posições possíveis. Sendo t o número de nós da nossa árvore, se todos os arrays de filhos estiverem lotados para todos os t nós, o sistema deve consumir $t \times 36$ espaços na memória. Comparando ambas as aplicações, o quesito memória se mostra o ponto negativo da utilização da Trie.

O compartilhamento de prefixos é uma forma de mitigar o consumo de memória durante a aplicação do sistema. Isso porque, seguindo essa configuração, em um mesmo nível da Trie não precisamos armazenar dois caracteres iguais em locais físicos diferentes apenas porque o caractere compõe outra chave de busca. Quando compartilhamos um mesmo caractere, além de reduzir o número de nós em cada nível da árvore e, conseqüentemente, a memória alocada a eles no sistema, garantimos que a complexidade de busca permaneça estritamente dependente do tamanho da chave de busca.

Cada um desses nós possui um array de filhos com tamanho *ALPHABET_SIZE* que dita quantos caracteres estamos considerando no sistema para a alocação de memória. Como estamos transformando o título para uma chave de busca apenas com letras minúsculas e números, o espaço alocado reduz de 62 (alfabeto maiúsculo + alfabeto minúsculo + números) para 36 (alfabeto minúsculo + números). Assim, o espaço ocupado por cada nó é proporcional ao tamanho do alfabeto que estamos considerando, que, na nossa aplicação, são 36 espaços que alocam 8 bytes $\approx$ 288 bytes.